

Testing Practices of AI Coding Agents in Open-Source Software

Anonymous Author(s)

Abstract

AI coding agents now contribute code to open-source projects, yet little is known about their testing practices in real-world development. We present the first large-scale empirical study of test contributions by AI coding agents, analyzing 33,580 pull requests (PRs) from five agents in the AIDev dataset. Our analysis reveals that 70.64% of agent PRs contain test files, though agents primarily contribute unit tests while advanced test types (integration, end-to-end, regression) remain rare. We find substantial variation in testing behavior across agents: Copilot produces the highest volume of test code with a test-to-code ratio (0.11), while Claude Code generates fewer tests but with the highest assertion density (11.93 assertions per test). OpenAI Codex, despite having the highest proportion of test-related PRs (96.24%), shows the lowest test-to-code ratio (0.03), suggesting quantity-over-depth behavior. These findings reveal a fundamental imbalance in agent testing contributions and highlight the need for human oversight in test design, particularly for integration and end-to-end scenarios. Our results provide actionable insights for practitioners selecting agents based on testing needs and inform future research on improving agent testing capabilities.

CCS Concepts

• **Software and its engineering** → **Software version control**; *Development frameworks and environments*.

Keywords

Pull requests, Code changes, AI agentic, software testing, mining software repositories, empirical study

ACM Reference Format:

Anonymous Author(s). 2025. Testing Practices of AI Coding Agents in Open-Source Software. In *Proceedings of MSR '26: Proceedings of the 23rd International Conference on Mining Software Repositories (MSR 2026)*. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

1 Introduction

AI coding agents now autonomously generate, modify, and submit code to open-source projects, fundamentally changing software development workflows [6, 11]. Tools like Claude Code produce widely accepted contributions, with 83.8% of agent-generated pull requests (PRs) merged into production projects [11], making systematic evaluation of agent-generated code quality essential.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

MSR 2026, Rio de Janeiro, Brazil

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
<https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

Existing research shows AI agents excel at code generation but face challenges in test generation. Zhang et al. [13] systematically reviewed 105 papers on LLMs for unit testing, revealing that while 60.5% of studies focus on test generation, significant challenges remain in generating comprehensive and effective test suites. Fakhoury et al. [2] demonstrated that interactive test-driven approaches can achieve 38.43% improvement in code generation accuracy, highlighting the importance of rigorous testing. However, empirical studies of AI agent frameworks reveal incomplete testing practices [4]. Hasan et al. found that deterministic components receive over 70% of testing effort while foundation model-based components receive less than 5%, indicating critical blind spots in current testing approaches. A work by Strandberg et al. [10] emphasizes the critical role of human responsibility in the design and generation of test cases, particularly for AI-based test automation systems. Despite this progress, little empirical evidence exists on how frequently AI agents contribute tests in real-world projects and whether different agents vary in their testing behavior. We address this gap by analyzing test contributions in agent-generated PRs from the AIDev dataset [6].

In this paper, we conduct an empirical study to analyze and understand test contribution patterns by AI coding agents using the AIDev dataset [6], which contains PRs from both AI agents and humans across thousands of open-source repositories. We investigate two research questions (RQs):

RQ1: *How frequently do coding agents contribute tests, and what types of tests are most common?*

RQ2: *How do different Agentic AIs vary in their test-related contributions?*

Our results indicate that 70.64% of agentic PRs include test files, but agents heavily favor unit tests (28,278 files) over end-to-end (3,637) and regression tests (1,260). Testing behavior varies by model: Copilot prioritizes test volume (test-to-code ratio 0.11), Claude Code emphasizes assertion depth (11.93 assertions per test), and Codex has high PR-level test usage (96.24%) but minimal patch-level contributions (0.03 test-to-code ratio). Agents thus handle unit testing well but leave major gaps in system-level coverage, so human oversight remains essential.

Contributions. (1) Quantitative analysis of test contribution patterns across five AI coding agents; (2) Characterization of test types generated by AI agents versus humans; (3) Comparative evaluation of testing behavior across different AI agents.

Paper organization. The paper is organized as follows: Section 2 presents our analysis methodology and results. Section 3 discusses implications. Section 4 discusses validity threats of our analyses. Section 5 reviews related work. Section 6 concludes the paper and suggests possible future work.

Replication package. Our data and scripts are available at [1].

2 Empirical Analysis and Results

2.1 Dataset

We used two types of data from the AIDev dataset [6] for our analysis: pull request metadata and commit-level details. Note that commit-level data contains contributions from both human and AI authors. When a PR is initially authored by an AI agent, we classify all test-related commits within that PR as AI contributions, regardless of the individual commit author. This approach captures the overall test coverage landscape in agent-generated PRs.

2.2 RQ1: How frequently do coding agents contribute tests, and what types of tests are most common?

2.2.1 Motivation. Understanding the frequency and types of tests contributed by AI coding agents is essential for assessing their role in software quality assurance. While prior work shows that agents excel at code generation but lag in test generation [8, 11], there is little empirical evidence on how often agents actually include tests in real-world contributions and which test types they prioritize. This knowledge helps practitioners set appropriate expectations for agent-generated code and identifies gaps where human intervention remains necessary.

2.2.2 Approach. To investigate how frequently coding agents contribute tests and what types of tests they produce, we analyzed agent-generated PRs from the AIDev dataset. We combined pull request metadata with commit-level file changes to examine actual agent contributions.

We identified test files using keyword-based heuristics common in mining software repositories research. Specifically, we searched for test-related indicators in three fields: filenames, commit messages, and patch content. Files were flagged as tests if they appeared in test-related directories, followed typical testing naming conventions (e.g., filenames ending in `.test.js` or `_test.py`, or located in `tests/` folders), or contained test-related keywords in their patches or commit messages. We initially analyzed only filenames and commit messages but found that including patch content significantly improved detection accuracy.

We then categorized identified test files into specific types: unit tests, integration tests, end-to-end tests, performance tests, benchmark tests, regression tests, and smoke tests. Finally, we computed descriptive statistics showing how many PRs included at least one test file and the distribution of test types across agent contributions.

2.2.3 Findings.

Agents frequently include tests, but nearly one-third skip testing entirely. Analyzing 33,580 AI-generated PRs revealed that 70.64% (23,722 PRs) contained at least one test file. While this demonstrates that most agent contributions engage with testing, the remaining 29.36% (9,858 PRs) contained no tests whatsoever. This substantial minority suggests that agents do not universally prioritize testing, even when making functional code changes.

Test contributions favor basic types while advanced testing remains rare. Table 1 reveals a pronounced imbalance in test type distribution. The largest category, "other" (214,418 files), consists of test files that lack standard naming conventions or classification

Table 1: Test Type Distribution

Test Type	Description	Count
Unit	Individual function/method validation	28,278
Integration	Component interaction verification	26,510
Performance	Speed and efficiency measurement	8,709
Benchmark	Standardized performance comparison	3,930
End-to-End	Complete user workflow validation	3,637
Regression	Previously fixed bug prevention	1,260
Smoke	Basic functionality sanity checks	355
Other	Unclassified or generic test files	214,418

markers. Among identifiable test types, unit tests dominate with 28,278 files, followed by integration tests (26,510 files). However, more sophisticated test types appear far less frequently: performance tests (8,709), benchmarks (3,930), end-to-end tests (3,637), regression tests (1,260), and smoke tests (355). This distribution indicates that agents primarily generate fundamental test types while rarely contributing to system-level or specialized testing scenarios.

Functional code vastly outnumbers test code. The 424,826 non-test files in agent PRs dwarf the test file count, revealing that agents prioritize functional implementation over comprehensive test coverage. This imbalance suggests that agents focus on delivering working code rather than ensuring robust validation through extensive testing.

Takeaway. Agents handle basic unit testing but fail to provide comprehensive coverage. Project maintainers should establish human oversight for integration, end-to-end, and regression testing, and enforce testing standards to address the 30% of agent PRs that contain no tests at all.

2.2.4 Validation.

Taking in consideration 10% margin of error we found a sample of 100 data. The sample were made unbiased by taking values that contained different types of test and by limiting the non test values. To assess the reliability of our automated test identification, two annotators independently labeled a stratified random sample of 100 file-level instances. Inter-annotator agreement was measured using Cohen's Kappa. We observed substantial agreement for both test presence ($\kappa = 0.66$) and test-type classification ($\kappa = 0.69$). These results indicate that test identification and categorization can be performed reliably using file names, commit messages, and code diffs alone.

RQ1 Summary: Test Contribution Frequency and Types

Over two-thirds of agent PRs include tests, but they heavily favor unit tests (28,278) over integration (26,510), end-to-end (3,637), and regression (1,260). This narrow focus leaves coverage gaps that still require human oversight.

2.3 RQ2: How do different Agentic AIs vary in their test-related contributions?

2.3.1 Motivation. Different AI agents are built with varying architectures, training data, and design goals, which may influence their testing behavior. Understanding these differences is crucial

Table 2: Human vs Agents Test Contribution

Contributor	Total PRs	Test PRs	Test PR (%)
Claude Code	459	355	77.34
Copilot	4,970	3,565	71.73
Cursor	1,541	509	33.03
Devin	4,827	3,375	69.92
OpenAI Codex	21,799	20,980	96.24
Human	6,618	2,695	40.72

for practitioners selecting agents for specific projects and for researchers seeking to improve agent capabilities. By comparing test contribution patterns across agents and against human baselines, we can identify which agents are more test-oriented, how their testing strategies differ in depth and breadth, and whether certain agents are better suited for quality-critical development contexts.

2.3.2 Approach. We address this RQ in two sub-questions for clearer analysis and comparison.

RQ2.1: How do AI agents vary in test contributions compared to humans?

We analyzed pull request metadata for both AI agents and humans in the AIDev dataset. Given that file-level differences were unavailable, we identified test-related PRs using keyword heuristics, searching titles and bodies for testing terms (e.g., "test", "unit test", "integration test", "pytest", "junit", "assert"). Any PR containing one of these terms was labeled test-related. Although the human and agent PR sets differed greatly in size, their proportions of test-related contributions differed notably.

RQ2.2: How do AI agents vary in test contributions among themselves?

We quantified differences in test-related contributions across agents using metrics derived from pull request and patch-level analysis. We measured the number of test files added or modified, the volume of test code changes (lines added and removed), and the ratio of test code to production code as a proxy for test coverage. We also analyzed test characteristics, including test types (e.g., unit, integration, performance) and assertion density to capture qualitative differences in testing behavior. We grouped all metrics by the agent that submitted each PR.

2.3.3 Findings.

Agents engage with testing more frequently than humans at the PR level. Among 6,618 human PRs, only 40.72% (2,695 PRs) were test-related, indicating humans often prioritize functional changes over testing as shown in table 2. In contrast, AI agents show substantially higher test engagement: OpenAI Codex leads at 96.24%, followed by Claude Code (77.34%) and Copilot (71.73%). All agents except Devin (52.94%) exceed the human baseline, suggesting agents are more consistently prompted or trained to include test-related changes.

Patch-level analysis reveals distinct testing philosophies across agents. Table 3 exposes fundamental differences in how agents approach testing. Copilot emphasizes volume, modifying the most test files (17,674) with the highest test-to-code ratio (0.11), but produces

shallow tests with only 1.76 assertions per test file. Claude Code takes the opposite approach: fewer test files (1,641) but the deepest validation logic with 11.93 assertions per test, indicating comprehensive verification within individual tests. Cursor and Devin occupy the middle ground with balanced volume and depth (8.38 and 5.02 assertions per test, respectively).

OpenAI Codex exhibits a PR-patch disconnect. Despite showing the highest PR-level test engagement (96.24%), Codex demonstrates the lowest patch-level test-to-code ratio (0.03) and moderate assertion density (4.09). This paradox suggests Codex frequently mentions or references tests in PR metadata without contributing substantial test code in patches.

Takeaway. Testing behavior is agent-dependent, reflecting differences in training data, design objectives, or prompting strategies. Practitioners should select agents based on testing needs: Copilot for high-volume test generation with strong test-code linkage, Claude Code or Cursor for assertion-rich validation logic, and recognize that high PR-level test engagement (like Codex) does not guarantee substantive test contributions at the patch level.

RQ2 Summary: Agent Testing Behavior Variation

Agent test-related PR rates (52.94%–96.24%) exceed the human baseline (40.72%). At the patch level, Copilot favors volume, Claude Code favors depth, whereas Codex shows high PR engagement but minimal patch-level contributions. Testing behavior varies by agent; practitioners should choose agents based on desired volume vs. depth.

3 Implications

Our empirical findings have important implications for practitioners integrating these tools into development workflows, AI agent developers designing next-generation coding assistants, and researchers advancing autonomous coding capabilities.

Implications for Project Maintainers. AI-generated PRs primarily contain unit tests, leaving significant coverage gaps in integration and end-to-end testing, often requiring human intervention. Agent testing behaviors vary, with some generating more tests or assertion-rich tests, highlighting the need for tailored agent selection and human oversight.

Implications for AI Agent Developers. The imbalance in test types shows agents primarily generate unit tests (26,510 vs. 28,278 total), with scarce end-to-end and regression tests, highlighting the need for developers to expand training data, refine prompting, and implement post-generation validation. Distinct behavioral differences across agents imply that design choices influence testing patterns, necessitating configurable behavior and transparency for effective agent utilization.

Implications for Researchers. The imbalance between unit and advanced test types and varying agent behaviors highlights opportunities to enhance agent support for integration/end-to-end testing, influenced by agent design. Future research should explore prompting/configuration effects on test generation and compare agent-generated tests with human-written ones in real-world settings.

Table 3: Test-related Code Changes by Coding Agent

Agent	Test Files	LOC Added	LOC Removed	Assertions	Test/Code	Assertions/Test
Claude Code	1,641	104,998	75,449	19,585	0.08	11.93
Copilot	17,674	469,167	328,639	31,085	0.11	1.76
Cursor	1,405	104,272	45,157	11,771	0.08	8.38
Devin	11,618	397,619	172,855	58,289	0.10	5.02
OpenAI Codex	5,674	233,540	42,753	23,234	0.03	4.09

4 Threats to Validity

Construct Validity. Our metrics capture test quantity and structure (e.g., test counts, test-to-code ratios, assertion density) but do not measure test quality or fault-detection effectiveness. Test-related PR identification based on keyword matching may include false positives or miss implicit test changes. Patch-level aggregation may also mask within-PR variation.

Internal Validity. Test identification and test-type classification rely on keyword-based heuristics applied to filenames, commit messages, and patches, which may miss non-standard tests or introduce misclassifications. We mitigate this by combining multiple signals and validating against ground-truth samples, and by focusing on aggregate trends rather than individual files. Attributing all commits in agent-generated PRs to agents may overestimate agent contributions, but this aligns with the AIDev dataset design and reflects PR-level practice.

External Validity. Our analysis is limited to the AIDev dataset and a fixed time window, which may not reflect current or future agent capabilities. The dataset focuses on open-source projects and five specific agents, limiting generalizability to other domains, agents, or newer model versions. The imbalance between agent and human PRs restricts direct statistical comparisons; we therefore emphasize within-agent patterns and qualitative differences.

Conclusion Validity. Statistical comparisons are affected by non-normal distributions and unequal sample sizes across agents. We use non-parametric tests where appropriate and focus on effect sizes and practical significance. Observed associations between agents and testing behavior are correlational, not causal, and may be influenced by agent design, user selection, or project context.

5 Related Work

AI Code Generation. Prior work evaluates AI coding tools and agent performance. Yetistiren et al. [12] showed that ChatGPT outperformed GitHub Copilot on HumanEval (65.2% vs. 46.3%). Li et al. [6] introduced the AIDev dataset of 456k agent-generated PRs, finding that agents are faster but their PRs are accepted less often than those by humans, indicating a trust gap. In another study by Li and Hassan of 567 GitHub pull requests generated by Claude Code finds that developers commonly use agents for tasks like refactoring, documentation, and testing and also indicates a high acceptance rate for agent-assisted PRs, though many still benefit from human revisions [7].

Test Generation with LLMs. Zhang et al. [13] reviewed 105 studies and found test generation to be the primary focus (60.5%). Key challenges include model adaptation and hybrid approaches. TiCoder [2] improved code accuracy by 38.4% through test-driven

interaction, while Mundler et al. [8] showed that multi-agent repair systems can outperform dedicated test generators. A systematic review by Garousi et al. provides insights into the capabilities and limitations of AI-powered test automation tools, based on an empirical evaluation with two open-source software systems [3].

Testing Practices in AI Agents. Hasan et al. [4] found that over 70% of testing targets deterministic components, while less than 5% focus on foundation models, leaving non-deterministic behavior largely undertested. Multi-agent frameworks such as AgentCoder [5] address this by separating roles for code and test generation. A research by Sami et al. introduces a multi-agent system where a testing agent is responsible for thoroughly evaluating software, generating and running unit and end-to-end tests [9].

Positioning. Unlike prior work, our work analyzes testing practices in real-world agent-generated PRs from AIDev. We provide the first systematic analysis of how agents write and integrate tests in production code, revealing agent-specific testing behavior.

6 Conclusion

We present a systematic analysis of 33,580 pull requests from five AI coding agents that reveals that while 70.64% of agent-generated PRs include tests, these are predominantly unit tests. This leaves significant coverage gaps, as integration, end-to-end, and regression tests are rarely contributed, often requiring human intervention. Testing behaviors differ significantly by agent. For example, Copilot generates the most tests relative to code, Claude Code creates the most assertion-dense tests, and Codex shows test-related PRs with minimal content. These variations indicate that AI model design, rather than consistent best practices, dictates agent testing behavior. Practically, agent selection should match testing goals: Codex for lightweight coverage, Copilot for test volume, and Claude Code or Cursor for richer validation logic. However, human oversight remains necessary to ensure broader test coverage, especially beyond unit tests. For research, our findings motivate work on reducing test-type imbalance, evaluating test effectiveness beyond surface metrics, and designing hybrid human-AI workflows for more robust testing.

Future work. Several directions merit further study. First, using mutation testing and fault detection to evaluate agent-generated tests can assess their effectiveness beyond syntax. Second, examining how prompting strategies affect test generation may yield more comprehensive suites. Finally, analyzing how agents maintain and update tests as code evolves would clarify their role in continuous development.

References

- [1] Anonymous. 2025. Testing Practices of AI Coding Agents in Open-Source Software. <https://anonymous.4open.science/r/MSR26-Mining-Challenge-AI-Agents-Testing>.
- [2] Sarah Fakhoury, Aaditya Naik, Georgios Sakkas, Saikat Chakraborty, and Shuvendu K. Lahiri. 2024. LLM-Based Test-Driven Interactive Code Generation: User Study and Empirical Evaluation. *IEEE Transactions on Software Engineering* (2024).
- [3] Vahid Garousi, Michael Felderer, Baki Karapınar, and Cemal Yilmaz. 2024. AI-powered Test Automation Tools: A Systematic Review and Empirical Evaluation. *Information and Software Technology* 167 (2024), 107321. doi:10.1016/j.infsof.2023.107321
- [4] Mohammed Mehedi Hasan, Hao Li, Emad Fallahzadeh, Gopi Krishnan Rajbahadur, Bram Adams, and Ahmed E Hassan. 2025. An Empirical Study of Testing Practices in Open Source AI Agent Frameworks and Agentic Applications. *arXiv preprint arXiv:2509.19185* (2025).
- [5] Dacheng Huang, Qi Bu, Jianming Zhang, Yuting Qing, Michael Luck, and Haipeng Cui. 2024. AgentCoder: Multi-Agent Code Generation with Effective Testing and Self-optimisation. *arXiv preprint arXiv:2312.13010* (2024).
- [6] Hao Li, He Zhang, and Ahmed E. Hassan. 2025. The Rise of AI Teammates in Software Engineering (SE) 3.0: How Autonomous Coding Agents Are Reshaping Software Engineering. *arXiv preprint arXiv:2507.15003* (2025).
- [7] Zhenhao Li and Ahmed E. Hassan. 2025. On the Use of Agentic Coding: An Empirical Study of Pull Requests on GitHub. *arXiv preprint arXiv:2509.14745* (2025). <https://arxiv.org/abs/2509.14745>
- [8] Niels Mündler, Mark Niklas Müller, Jingxuan He, and Martin Vechev. 2024. Code agents are state of the art software testers. In *ICML 2024 Workshop on LLMs and Cognition*.
- [9] Md. A. Sami, Md. Saidur Rahman, and Ahmed E. Hassan. 2024. Experimenting with Multi-Agent Software Development: Towards a Unified Platform. *arXiv preprint arXiv:2406.05381* (2024). <https://arxiv.org/abs/2406.05381>
- [10] Kristian Strandberg, Vahid Garousi, and Michael Felderer. 2025. Ethical Challenges and Software Test Automation. *Software Quality Journal* (2025). https://www.researchgate.net/publication/394538598_Ethical_challenges_and_software_test_automation Early access.
- [11] Miku Watanabe, Hao Li, Yutaro Kashiwa, Brittany Reid, Hajimu Iida, and Ahmed E Hassan. 2025. On the use of agentic coding: An empirical study of pull requests on github. *arXiv preprint arXiv:2509.14745* (2025).
- [12] Burak Yetistiren, Isik Özsoy, Miray Ayerdem, and Eray Tüzün. 2023. Evaluating the Code Quality of AI-Assisted Code Generation Tools: An Empirical Study on GitHub Copilot, Amazon CodeWhisperer, and ChatGPT. *arXiv preprint arXiv:2304.10778* (2023).
- [13] Quanjun Zhang, Chunrong Fang, Siqi Gu, Ye Shang, Zhenyu Chen, and Liang Xiao. 2025. Large Language Models for Unit Testing: A Systematic Literature Review. *arXiv preprint arXiv:2506.15227* (2025).